

Software Requirements Specification

User Posts and Albums Viewer

Document Version	SRS v0.1
Status	Draft
Prepared By	Atul Sharma
Date	04 October 2026
Technology	HTML5, CSS3, JavaScript (ES6+), jQuery 4.0.0
Data Source	JSONPlaceholder REST API

This document defines the testable functional and non-functional requirements for the application.

1. Introduction

1.1 Purpose

This SRS defines the testable functional and non-functional requirements for the User Posts and Albums Viewer.

1.2 Product Scope

The system shall retrieve users, display the requested first-name + last-name value, allow selection of a user, retrieve that user's Posts and Albums from separate endpoints, and display both datasets simultaneously in separate UI sections. Loading, error, and empty states are required.

2. Overall Description

The application is a presentation-focused client-side website. JSONPlaceholder acts as the ready-made external API/data layer. There is no custom backend, application-owned business/service layer, database, authentication system, or persistent storage.

2.1 Product Perspective

Component / Layer	Responsibility
Presentation layer	HTML/CSS structure and visual presentation of users, selected-user area, Posts panel, Albums panel, loading, error, and empty state
Client-side application logic	jQuery event handling, API communication, response processing, state, and DOM updates
External API/data layer	Provides user, Post, and Album records through REST-style GET endpoints

2.2 Assumptions

- JSONPlaceholder is the required external API/data source.
- The user-facing name is based on the API `name` field.
- Posts and Albums are separate API resources and separate requests.
- Posts and Albums must be visible simultaneously in separate UI sections.
- No custom backend/business layer or database is required.

2.3 Constraints

- HTML and jQuery are explicitly required technologies.
- User, Post, and Album data shall come from JSONPlaceholder.
- Loading and error handling are required.
- Posts means `/posts`, not `/photos`.

3. Functional Requirements

FR-01 - Retrieve Users

The application shall retrieve the list of users from `https://jsonplaceholder.typicode.com/users` using HTTP GET.

FR-02 - Display Users

The application shall display each retrieved user using the API `name` value as the requested first-name + last-name display value.

FR-03 - Select User

The application shall provide an interactive mechanism to select one displayed user.

FR-04 - Retrieve Posts

After a user is selected, the application shall retrieve Posts associated with the selected user ID from `https://jsonplaceholder.typicode.com/users/{userId}/posts`.

FR-05 - Retrieve Albums

After a user is selected, the application shall retrieve Albums associated with the selected user ID from `https://jsonplaceholder.typicode.com/users/{userId}/albums`.

FR-06 - Present Posts and Albums Together

The application shall present Posts and Albums in two separate UI sections and shall display both sections at the same time. Posts and Albums shall not be combined into a single list.

FR-07 - Loading State

The application shall display appropriate loading feedback while required API data is being retrieved.

FR-08 - Error Handling

The application shall handle API/request failures and display an appropriate user-facing error message.

FR-09 - Dynamic Update

Selecting another user shall update the related content without requiring a manual full-page refresh.

4. External Interface Requirements

4.1 API Interface

ID	Method	Complete URL	Purpose
API-01	GET	https://jsonplaceholder.typicode.com/users	Retrieve users
API-02	GET	https://jsonplaceholder.typicode.com/users/{id}/posts	Retrieve selected user Posts
API-03	GET	https://jsonplaceholder.typicode.com/users/{id}/albums	Retrieve selected user Albums

4.2 User Interface Requirements

- Page header identifying the application.
- Users area showing the requested name value.
- Clear indication of the selected user.
- A Posts section and an Albums section visible simultaneously.
- Loading indication during data retrieval.
- User-facing error state when an API request fails.
- Empty-state message when either returned dataset is empty.

5. Use Case

UC-01: Browse Users and View Posts + Albums

Primary Actor: User

Goal: Select a user and view the selected user's Posts and Albums simultaneously.

Main Flow

1. Open application.
2. Retrieve users.
3. Display users.
4. Select a user.
5. Start the Posts and Albums requests independently.
6. Receive the responses.
7. Render Posts and Albums in separate sections at the same time.

Alternative Flow

If an API request fails, show the appropriate error state. If data is being retrieved, show loading feedback.

6. Detailed Behavioral Requirements

6.1 Initial Page Load

- Initialize the user-list area.
- Request the users resource.
- Show loading feedback while the request is in progress.
- On success, render the user `name` value.
- On failure, show the users error state.

6.2 User Selection

- A displayed user shall be selectable.
- The selected user ID shall determine the Posts and Albums requests.
- The UI shall indicate the selected user.

6.3 Posts Section

- The Posts section shall use the selected user ID.
- It shall display only Posts content.
- It shall remain visible beside the Albums section.

6.4 Albums Section

- The Albums section shall use the selected user ID.
- It shall display only Albums content.
- It shall remain visible beside the Posts section.

6.5 Loading, Error, and Empty States

- Loading feedback shall communicate request progress.
- Errors shall communicate which required operation failed where practical.
- Empty results shall display a meaningful empty-state message.

7. Non-Functional Requirements

ID	Requirement	Expected Outcome
NFR-01	Usability	User selection and simultaneous Posts/Albums viewing are straightforward.
NFR-02	Responsiveness of feedback	The UI indicates when data is loading.
NFR-03	Error handling	Failures are handled gracefully with meaningful feedback.
NFR-04	Maintainability	API communication, state, interaction, and rendering remain logically separated.
NFR-05	Browser compatibility	Works in commonly used modern browsers.

8. Acceptance Criteria

ID	Acceptance Criterion
AC-01	Opening the application requests the users resource.
AC-02	The user list displays the API name value as the requested first-name + last-name display.
AC-03	A user can be selected.
AC-04	The selected user ID is used for both related endpoints.
AC-05	The selected user's Posts are retrieved from the Posts endpoint.
AC-06	The selected user's Albums are retrieved from the Albums endpoint.
AC-07	Posts and Albums are shown in separate sections at the same time.
AC-08	Loading feedback is shown while required data is being retrieved.
AC-09	An appropriate error state is shown when a required API request fails.
AC-10	Selecting another user updates the related Posts and Albums without full-page refresh.

9. Out of Scope

- Custom backend development.
- Application-owned business/service layer.
- Custom database or persistent storage.
- Authentication, authorization, registration, or login.
- CRUD operations against an application-owned store.
- Administrative features.
- Advanced UI/UX beyond a clean presentation layer.

10. Requirement Traceability

Requirement Area	Derived From	Verification Artifact
User retrieval/display	FR-01, FR-02	TC-01, TC-02
User selection	FR-03	TC-03
Posts retrieval/display	FR-04, FR-06	TC-04, TC-07
Albums retrieval/display	FR-05, FR-06	TC-05, TC-07
Loading state	FR-07	TC-06
Error handling	FR-08	TC-08
Dynamic update	FR-09	TC-09

11. Status

SRS v0.1 - Draft.